Chapter # 03

Date:

Solving Problems by Searching :-

3.1 Explain why problem formulation must follow goal formulation.

Sol Goal formulation decides what the agent wants to achieve, while problem formulation decides how that goal will be represented and solved using states, actions and costs.

Therefore, the goal must be known first because it tells us which details of the real world are relevant

For example, if the goal is to travel from Arad to Bucharest, the problem can be formulated as:

- Initial state : Arad
- Actions : Travel to neighboring cities
- Goal state : Bucharest
- Path cost : Distance travelled

Thus, goal formulation comes first because the goal determines what problem needs to be solved.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

3.2 Your goal is to navigate a robot out of a maze. The robot starts in the center of the maze facing north. You can turn the robot to face N, E, S, W. You can direct the robot to move forward a certain distance, although it will stop before hitting a wall.

a) Formulate this problem. How large is the state space?

A state can be represented as:

(x, y, orientation)

where (x, y) is robot's position and orientation is N, S, E or W.

- Initial state: Center of maze, facing north.
- Actions: Turn N, S, E, W or move forward chosen distance.
- Transition Model: Changes the robot's position or direction according to the action.
- Goal test: Robot has reached an exit.
- Path cost: Distance travelled or number of actions.

Since the robot can stop at any position along a corridor, there can be infinitely many possible positions, so the original state space is infinite.

PAPER PRODUCT

Scanned with CamScanner

Date:

b) In navigating a maze, the only place we need to turn is at the intersection of two or more corridors. Reformulate this problem using this observation. How large is the state space now?

Ans We only need to represent the robot when it is at a turning point or intersection. If there are N such points and four possible orientations, the state space is at most:

$$\boxed{4N}$$

This changes the original infinite state space into a finite state space.

c) From each point in the maze, we can move in any of the four directions until we reach a turning point, and this is the only action we need to do. Reformulate the problem using these actions. Do we need to keep track of the robot's orientation now?

Ans A state can now simply represent the current turning point. An action means moving directly from one turning point to another reachable turning point.

Therefore, orientation does not need to be stored because the chosen movement already determines the direction. If there are N turning points:

$$\boxed{\text{State space} = N}$$

SP PAPER PRODUCT

Scanned with CamScanner

Date:

d) In our initial description of the problem we already abstracted from the real world, restricting actions and removing details. List 3 such simplifications we made.

Ans Three simplifications are:

1. The robot's physical size and shape are ignored.
2. Its movements and turns are assumed to be perfectly accurate.
3. The maze is assumed to be fixed; walls and passages do not change.

3.3 Suppose two friends live in different cities on a map. On every turn, we can simultaneously move each friend to neighboring city. The friend who arrives first must wait for the other before the next turn begins. We want the two friends to meet as quickly as possible.

a) Write a detailed formulation for this search problem.

Ans Let the state be: (i, j) where i is the city of friend 1 and j is the city of friend 2.

- Initial state: (i_0, j_0), their starting cities.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

- Actions: Both friends simultaneously move to neighboring cities.
- Transition model: $(i, j) \rightarrow (i', j')$
- Goal test: Both friends are in the same city: $i = j$
- Step cost: Since the faster friend waits, the time for one turn is:

$$\max(d(i, i'), d(j, j'))$$

The objective is to minimize the total meeting time.

b) Let $D(i, j)$ be the straight-line distance between cities i and j. Which heuristic functions are admissible? i) $D(i, j)$ ii) $2 \cdot D(i, j)$ iii) $D(i, j)/2$

Ans The admissible heuristic is: $D(i, j)/2$
Because both friends are moving toward each other simultaneously, so the minimum possible meeting time can be approximately half their straight-line separation.

$D(i, j)$ and $2 \cdot D(i, j)$ may overestimate the actual minimum cost, so they are not always admissible.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

c) Are there completely connected maps for which no solution exists?

Ans Yes. Consider a completely connected map containing only two cities. Each friend starts in a different city. On every turn they must move, so they simply exchange cities and never occupy the same city at the same time.

d) Are there maps in which all solutions require one friend to visit the same city twice?

Ans Yes. In some maps, one friend may have to travel through a loop and revisit a city so both friends arrive at the meeting city on the same turn. This happens because neither friend can simply remain in a city waiting between turns.

3.4 Show that the 8-puzzle states are divided into two disjoint sets, such that any state is reachable from any other state in the same set, while no state is reachable from any state in the other state. Devise a procedure to decide which set a given state is in, and explain why this is useful for generating random states.

PAPER PRODUCT

Scanned with CamScanner

Date:

Ans The two sets can be identified using the number of inversions in a puzzle state. First write the tiles as a sequence from left to right and top to bottom, ignoring the blank squares. An inversion occurs when a large-numbered tile appears before a smaller-numbered tile. For example:

1, 2, 3, 4, 5, 6, 8, 7

contains one inversion: 8, 7

For a 3×3 8-puzzle:

- states with an even number of inversions belong to one set,
- states with an odd number of inversions belong to the other set.

A legal move never changes this parity. Therefore, a state from one set can never reach a state in the other set. Since there are:

$$9! = 362,880$$

possible arrangements, each set contains:

$$\boxed{\frac{9!}{2} = 181,440} \text{ states}$$

This is useful when generating random puzzles because we can check the inversion parity & generate only states that are actually solvable from the chosen goal state.

SP PAPER PRODUCT

Scanned with CamScanner

Date: _____

3.5 Consider the n-queens problem using the "efficient" incremental formulation given on page 72. Explain why the state space has atleast $\sqrt[3]{n!}$ states and estimate the largest n for which exhaustive exploration is feasible

Ans In the efficient formulation, queens are placed one column at a time, and a queen is placed only in a square that is not attacked by an already placed queen.

If k queens have already been placed, each can attack at most three squares in the next column: one horizontally and two diagonally. Therefore, atleast:

$n - 3k$ positions may remain available.

The number of possible partial states is therefore atleast approximately:

$$n(n-3)(n-6)\ldots$$

The factor of n! can be divided into three similar groups:

$$n(n-3)(n-6)\ldots$$
$$(n-1)(n-4)(n-7)\ldots$$
$$(n-2)(n-5)(n-8)\ldots$$

PAPER PRODUCT

Scanned with CamScanner

Date:

Their product is $n!$. The first group is the largest, therefore it is atleast: $\sqrt[3]{n!}$

Even this lower bound grows extremely quickly. For example:

$$\sqrt[3]{30!} \approx 6.4 \times 10^{10} \text{ states}$$

Hence exhaustive exploration becomes impractical for n around the mid-20s to about 30, depending on the speed of the computer.

3.6 Give a complete problem formulation for each of the following. Choose a formulation that is precise enough to be implemented.

a) Using only 4 colors, you have to color a planar map in such a way that not two adjacent regions have the same color.

- State: A partial assignment of colors to regions.
- Initial state: No region is colored.
- Actions: Assign one of four colors to an uncolored region without conflicting with an already colored neighbor.
- Transition Model: The selected region receives the chosen color.
- Goal test: Every region is colored and no two adjacent regions have the same color.
- Path cost: One unit per coloring action.

PAPER PRODUCT

Scanned with CamScanner

Date: _____

b) A 3-foot-tall monkey is in a room where some bananas are suspended for the 8-foot ceiling. He would like to get the bananas. The room contains two stackable, movable, climbable 3-foot-high crates.

- State: Monkey position, position of both crates, whether creates are stacked, and whether the monkey is on the floor or a crate.
- Initial state: Monkey and crates at their starting positions; monkey does not have the bananas.
- Actions: Walk, push a crate, stack creates, climb a crate, and grab bananas.
- Transition model: Updates the position of the monkey and crates according to the action.
- Goal test: Monkey has obtained the bananas.
- Path cost: One unit per action.

The monkey must eventually move the creates below the bandanas, stack them and climb high enough to reach the bananas.

c) You have a program that outputs the message "illegal input record" when fed a certain file of input records. You know that processing of each record is independent of the other records. You want to discover what record is illegal.

- State: Set of records that could still contain the illegal record.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

- Initial State: All input records are possible candidates
- Actions: Test a record or a group of records using the program.
- Transition model: If the error appears, the illegal record is in the tested group; otherwise it is in the remaining group.
- Goal test: Only one possible illegal record remains.
- Path cost: Number of tests performed.

Dividing the records into groups can reduce the number of tests required.

d) You have three jugs, measuring 12 gallons, 8 gallons, 3 gallons, and a water faucet. You can fill the jugs up or empty them out from one to another or onto the ground. You need to measure out exactly one gallon.

Ans Represent a state as: (x, y, z) where x, y, z contain the amounts of water in the 12, 8, 3 gallon jugs.

- Initial state: (0,0,0)
- Actions: Fill a jug, empty a jug, or pour water from one jug to another.
- Transition model: Changes the amount of water in the affected jugs.
- Goal test: Any jug contains exactly: 1 gallon.
- Path cost: One unit per action.

PAPER PRODUCT

Scanned with CamScanner

Date:

3.7 Consider the problem of finding the shortest path between two points on a plane that has convex polygonal obstacles, as shown in Fig 3.31.

a) Suppose the state space consists of all positions (x, y) in the plane. How many states are there? How many paths are there to the goal?

Ans Since x any y can take continuous real values, there are infinitely many possible states. There are also infinitely many possible paths from the start to the goal. This continuous representation is therefore unsuitable for ordinary search.

b) Explain briefly why the shortest path from one polygon vertex to any other must consist of straight-line segments joining some of the vertices of the polygons. Define a good state space now. How large is this state space?

Ans If shortest path should be a straight line whenever no obstacle blocks it. If a path bends in free space, the bend can be replaced by a shorter straight segment.

Therefore, bends are necessary where only obstacles force them, which occurs at polygon vertices.

PAPER PRODUCT

Scanned with CamScanner

Date:

A better state space therefore contains:

- the start point S;
- the goal point G_T;
- all vertices of the polygonal obstacles.

If there are V obstacle vertices:

$$\text{State-space size} = V + 2$$

c) Define the necessary functions to implement the search problem. Use the straight-line distance for the heuristic function.

- Initial state: S
- Actions(v): Move from vertex v to any other visible vertex that can be reached by a straight line without crossing an obstacle.
- Result(v, a): The vertex reached by that action.
- Goal test: Current state is G_T.
- Step cost: Straight-line distance between the two vertices.
- Heuristic: $h(v)$ = straight-line distance from v to G_T.

d) Apply one or more algorithms in this chapter and comment on their performance.

Ans A* search is a suitable algorithm because it uses the straight-line-distance heuristic to guide the search toward the goal.

PAPER PRODUCT

Scanned with CamScanner

Date: _____

Uniform-cost search can also find the shortest path, but it usually explores more states because it has no heuristic guidance.

Breadth-first search is less suitable because different path segments can have different distances.

3.8 On page 68, we said that we would not consider problems with negative path costs. In this exercise, we explore this decision in more depth.

a) Suppose that actions can have arbitrarily large negative costs; explain why this possibility would force any optimal algorithm to explore the entire state space.

If negative costs are allowed, a path that initially looks expensive may later contain an action with a very large negative cost and become the cheapest path.

Therefore, after finding a goal, the algorithm cannot safely assume that it is optimal because an unexplored path may have a lower total cost.

Hence, an optimal algorithm may need to explore the entire state space.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

b) Does it help if we insist that step costs must be greater than or equal to some negative constant c? Consider both trees and graphs.

Ans Not in general. In a tree, a sufficiently long path containing negative-cost steps can continue reducing the total path cost.

In a graph, a negative-cost cycle can be repeated again and again, reducing the total cost without limit.

A finite graph can have a well-defined optimal solution only when harmful negative-cost cycles are excluded.

c) Suppose a set of actions forms a loop and all these actions have negative cost. What does this imply about optimal behavior?

Ans Every time the agent completes the loop, its total path cost becomes smaller.
Therefore, the agent could repeat the loop indefinitely:

$$C, C-k, C-2k, C-3k, \ldots.$$

and there would be no finite optimal solution.

d) Why do humans not drive around scenic loops indefinitely, and how can an artificial agent avoid this?

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Ans. Humans consider more than scenary. Repeating the loop also costs time, fuel, energy and delays reaching the destination.

An AI system should therefore use a complete cost function that includes these factors. Its state may also include information such as time, fuel or previous visits.

With a realistic performance measure, repeatedly driving around the same loop will eventually become undesirable.

e) Can you think of a real domain in which step costs are such as to cause looping?

Ans. One example is a profitable trading or currency-arbitrage cycle. If completing a cycle produces more profit than it cost, & cost is represented as negative profit, repeatedly the cycle would continue decreasing the total cost.

In reality, limits such as changing prices, transaction fees and available money prevent infinite repetition.

3.9 Three missionaries and three cannibals are on one side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side

PAPER PRODUCT

Scanned with CamScanner

Date:

without ever leaving missionaries outnumbered by cannibals.

a) Formulate the problem precisely and draw the complete state space.

Represent each state as: (M, C, B)

where:

- M = missionaries on the left bank;
- C = cannibals on the left bank;
- B = side containing the boat, L or R.

Initial state: (3, 3, L)

Goal state: (0, 0, R)

Possible boat loads are: (1M), (2M), (1C), (2C), (1M, 1C)

A state is legal only when, on either bank, missionaries are never outnumbered by cannibals whenever missionaries are present.

PAPER PRODUCT

Scanned with CamScanner

Date: _____

where:

A = (3,3,L)	I = (2,2,L)
B = (3,2,R)	J = (0,2,R)
C = (3,1,R)	K = (0,3,L)
D = (2,2,R)	L = (0,1,R)
E = (3,2,L)	M = (0,2,L)
F = (3,0,R)	N = (1,1,L)
G = (3,1,L)	O = (0,0,R)
H = (1,1,R)	

b) Implement and solve the problem optimally using an appropriate search algorithm. Is it a good idea to check for repeated states?

Ans Breadth-First Search (BFS) is suitable because every river crossing can be treated as having equal cost, and BFS finds the solution requiring the fewest crossings.

One optimal solution is:

2 Cannibals cross →
1 Cannibal returns ←
2 Cannibals cross →
1 Cannibal returns ←
2 Missionaries cross →
1 Missionary + 1 Cannibal return ←
2 Missionaries cross →
1 Cannibal returns ←

SP PAPER PRODUCT

Scanned with CamScanner

Date:

2 Cannibals cross →
1 Cannibal returns ←
2 Cannibals cross →

Everyone is now on the opposite bank.

Yes, repeated-state checking is important because many moves can return the problem to a previously visited state. Without checking, the search can repeatedly explore the same situations.

c) Why do ~~we~~ people have difficulty solving this puzzle even though the state space is simple?

People often try to make every move appear to take immediate progress toward the goal. However, solving this puzzle sometimes requires bringing people back across the river, which appears to undo progress.

Search algorithms handle this more systematically by exploring legal states instead of depending only on human intuition.

3.10 Define in your own words the following terms: state, state space, search tree, search node, goal, action, transition model, and branching factor.

PAPER PRODUCT

Scanned with CamScanner

Date:

1. **State**: A state is a representation of the current situation of the problem at a particular point.

2. **State Space**: The state space is the set of all states that can be reached from the initial state by performing valid actions.

3. **Search Tree**: A search tree is the tree generated while searching for a solution. The root is the initial state, branches represent actions, and child nodes represent resulting states.

4. **Search Node**: It is a data structure used by a search algorithm. It usually stores the state, parent node, action and path cost.

5. **Goal**: A goal describes the state or condition that the agent is trying to achieve.

6. **Action**: An action is an operation available to the agent that can change the current state.

7. **Transition Model**: It describe what new state results when an action is performed in the current state. Result(s, a) = s'

8. **Branching Factor**: is the number of successor nodes that can be generated from a node. A larger branching factor usually makes the search problem harder because more alternatives must be considered.

SP PAPER PRODUCT

Scanned with CamScanner

Chapter #04

Date:

Beyond Classical Search :-

4.1 Give the name of the algorithm that results from each of the following special cases:

a) Local beam search with $k = 1$.

Ans Hill-climbing search.

With only one state retained, local search becomes hill climbing because only the best current state is kept.

b) Local beam search with one initial state and no limit on the number of states retained.

Ans Breadth-first search.

Without a limit on the number of retained states, all states at each level are retained, which gives the behavior of breadth-first search.

c) Simulated annealing with $T = 0$ at all times (and omitting the termination test).

Ans Hill-climbing search.

At $T = 0$, worse moves are never accepted, so the algorithm always moves only toward better states.

PAPER PRODUCT

Scanned with CamScanner

Date:

d) Simulated annealing with $T = \infty$ at all times.

Ans Random walk.

At infinite temperature, moves are accepted without considering whether they improve or worsen the current state, producing random movements through the state space.

e) Genetic algorithm with population size $N = 1$.

Ans Randomized hill climbing.

With only one individual, there is no meaning-ful crossover between two different parents, so the search behaves like a stochastic local search using mutation.

4.2 Exercise 3.16 considers the problem of building railway tracks under the assumption that pieces fit exactly with no slack. Now consider the real problem, in which pieces don't fit exactly but allow for up to 10 degrees of rotation to either side of the "proper" alignment. Explain how to formulate the problem so it could be solved by simulated annealing.

Ans The problem can be formulated as a continuous optimization problem in which

SP PAPER PRODUCT

Scanned with CamScanner

Date:

the rotation angle of each track piece is treated as a variable.

A state represents a complete arrangement of the railway track, including the rotation angle of each piece.

- State: The positions and rotation angles of all track pieces.
- Initial state: An initial arrangement of the track pieces.
- Successor: Change the rotation angle of one or more pieces by a small amount.
- Constraint: Each piece can rotate upto 10° in either direction from its proper alignment.
- Objective function: Measure how well the track pieces connect, giving higher values to smoother and better-connected tracks.

Simulated annealing starts with an initial arrangement and repeatedly makes small changes. It normally accepts better arrangements but sometimes accepts worse arrangements, especially at high temperature. As the temperature decreases, the algorithm becomes less likely to accept worse arrangements and gradually settle on a good ~~condition~~ solution.

Thus, simulated annealing can be used to find a suitable set of rotation angles even when the pieces do not fit exactly.

PAPER PRODUCT

Scanned with CamScanner

Date:

4.3 In this exercise, we explore the use of local search methods to solve TSPs of the type defined in Ex 3.30.

a) Implement and test a hill-climbing method to solve TSPs. Compare the results with optimal solutions obtained from the A* algorithm with the MST heuristic (Exercise 3.30).

Ans For a Travelling Salesperson Problem (TSP), a state can be represented as an ordered tour of all cities.

A hill-climbing solution can be implemented as follows:

1. Start with a random tour.
2. Generate neighbouring tours by changing the order of cities, such as swapping 2 cities.
3. Calculate the total distance of each neighbouring tour.
4. Move to the neighbour with the lowest distance.
5. Stop when no neighbour provides an improvement.

The resulting tour is then compared with the optimal tour obtained using A* with the MST heuristic.

Hill-climbing normally requires much less search than exhaustive methods, but it can stop at a local optimum, so its solution may not always be optimal.

PAPER PRODUCT

Scanned with CamScanner

b) Repeat part (a) using a genetic algorithm instead of hill climbing.

For a generic algorithm:

1. Generate a population of random TSP tours.
2. Calculate the fitness of each tour.
3. Select better tours as parents.
4. Apply crossover to create new tours.
5. Apply mutation to introduce small changes.
6. Repeat for several changes.
7. Return the best tour found.

The genetic algorithm's result is also compared with the optimal A* solution.

Genetic algorithms can explore many different parts of the search space because they work with a population of solutions, but they do not guarantee an optimal solution.

4.4 Generate a large number of 8-puzzle and 8-queens instances and solve them (where possible) by hill climbing (steepest-ascent and first-choice variants), hill climbing with random restart, and simulated annealing. Measure the search cost and percentage of solved problems and graph these against the optimal solution cost. Comment on your results.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Ans The experiment compares four local-search methods:

1. Steepest-ascent hill climbing
2. First-choice hill climbing
3. Hill climbing with random restart
4. Simulated annealing.

For each method, many random 8-puzzle and 8-queens instances should be generated. For every instance, record:

- Search cost
- Whether the problem was solved
- Optimal solution cost.

The results can then be plotted with optimal solution cost on the x-axis and search cost or percentage solved on the y-axis.

Expected Observations :-

Simple hill climbing can become stuck in local maxima/minima, ridges, or plateaux, so it may fail even when a solution exists.

Random-restart hill climbing improves the success rate because it starts again from different initial states.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Simulated annealing can escape local optima because it sometimes accepts worse moves, particularly when the temperature is high.

Therefore, the experiments show that adding randomness and multiple starting points can significantly improve the ability of local-search methods to find solutions.

Q: What is a Heuristic Search? Explain with examples

Ans: Heuristic search is a search method that uses additional knowledge about the problem to decide which states are more promising and should be explored first.

A heuristic function, written as $h(n)$, estimates the cost or distance from a current state n to a goal state.

Instead of exploring states blindly, heuristic search uses this estimate to guide the search toward the goal.

Example: Route Finding

Suppose we want to travel from Karachi to Islamabad. The straight-line distance between the current city and Islamabad can be used as a heuristic. A city that appears closer to the destination can be explored first.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

Example: 8-Puzzle

A common heuristic is the number of misplaced tiles. A state with fewer misplaced tiles is considered closer to the goal.

Q: What is Brute-Force search? Explain with examples.

Ans: Brute-force search is a problem-solving approach that systematically explores possible solutions without using special problem-specific knowledge to guide the search.

It relies mainly on the defined search space and checks possible states or solutions until a solution is found.

Examples:-

1. Password Cracking: If a password has 4-digits, a brute force method can try:

   0000, 0001, ..., 9999

   until the correct password is found.

2. Travelling Salesperson Problem: A brute-force method can generate every possible tour of the cities, calculate the distance of each tour and select the shortest one.

SP PAPER PRODUCT

Scanned with CamScanner

Date:

3. 8-Puzzle: A search algorithm such as breadth-first search can systematically explore states without using a heuristic to decide which state looks more promising.

* Brute-force methods can be simple and complete, but they may require a very large amount of time and memory when the search space is large.

PAPER PRODUCT

Scanned with CamScanner